GrantBind: Task-Scoped Tool Execution Mediation against Indirect Prompt Injection

Anonymous Authors

***Abstract*—Tool-using language-model agents encounter untrusted content while carrying out tasks with side effects. GrantBind mediates tool calls with task-scoped grants, action audits using bounded evidence, and one-use execution tickets. We compare recorded baseline and defended runs on four AgentDojo suites. On 1,943 paired Qwen3.7-Flash cases, task success rises from 73.3% to 79.3% and native attack hits fall from 438 to zero. On 1,836 DeepSeek-Flash cases, hits fall from 38 to 5 and task success changes from 83.6% to 82.4%. On a separate 108-case grid, Qwen3.8-Flash with GrantBind v3.7 completes 98 tasks versus 91 at baseline, with two native hits in each arm on different keys. Qwen3.5-Flash with same-model auditing loses task utility under attack, from 84.4% to 54.9%. On 97 clean tasks, its undefended and same-model configurations complete 90 and 53 tasks, respectively. A mixed configuration with Qwen3.7-Flash as auditor and a revised L2 output protocol completes 82. The clean comparison captures the joint change in configuration.**

***Index Terms*—indirect prompt injection, LLM agents, tool authorization, execution mediation, AgentDojo**

I. Introduction

A. Background

Language-model agents read emails, documents, and messages before choosing which tools to invoke. An indirect prompt injection embeds instructions in those external sources to redirect the agent [1], [2]. Consider a user who asks an agent to summarize a Slack channel and send the summary to a colleague. The channel history supplies facts for the summary, while an injected message can attempt to change the recipient or add unrelated material to the outgoing message. AgentDojo evaluates legitimate task completion and attacker objectives in this setting [3].

B. Problem and Challenges

The problem is to constrain the effects of tool use while preserving the agent's ability to gather facts and act on them. The first challenge is to preserve the distinction between instructions and evidence when both appear in the planning history. The second is to carry task-level authority down to an individual call. The same messaging tool can send a legitimate summary or disclose unrelated data, depending on the recipient and body. The third is to enforce the decision on the proposed batch and preserve the native order of permitted calls. Overly restrictive authorization or audit can also stop the legitimate task, so task success and attack success both matter.

C. Approach

GrantBind uses the trusted request to compile task-scoped tool grants. It reconstructs a source-aware planning view and retains raw returns for audit. For each candidate batch, the host checks grants and native schemas, then sends the non-exempt calls to a semantic auditor. The reference monitor binds an approved batch to its decision and issues a one-use ticket for each permitted call.

We document this execution boundary and compare native task and attack outcomes on paired AgentDojo keys within each model. The Qwen3.5-Flash study separates protocol failures, logged blocks, and other task losses, and measures how much clean utility returns in a mixed configuration.

II. Threat Model and Scope

We consider a single text-based agent with a trusted user request and host-provided tool catalogue. The attacker can influence supported tool-return content, but not the trusted request, tool definitions, or mediation code. The target failure is an action or answer redirected away from the authorized task.

GrantBind enforces the tool-call boundary. Retrieved facts, object-level access, and parameter semantics depend on the available evidence and the model's judgment. Final text follows the native generation path and can contribute to the benchmark's attack predicate. Task authority comes from the trusted request, including bounded task files explicitly delegated by the user.

III. GrantBind Design

Fig. 1 shows the execution path.

A. L1: Isolated Planning View

GrantBind reconstructs the planning history with source wrappers around tool returns. The original returns remain available for audit and tracing. An optional projection for banking monthly-spending queries omits the subject field and preserves row order and structural fields. The recorded evaluation uses source wrapping with projection disabled.

B. L2: Static Task and Tool Authorization

When action authorization is required, the main model first extracts a static task view and then compiles global tool permissions against the complete trusted request and native tool definitions. The host resolves span identifiers to literal request text, merges registered automatic reads, and freezes the grants and catalogue for the task. The view exposes references, order, and conditions to planning and audit. Invalid authorization outputs fail closed. An unauthorized or schema-invalid candidate blocks the entire batch before L3.

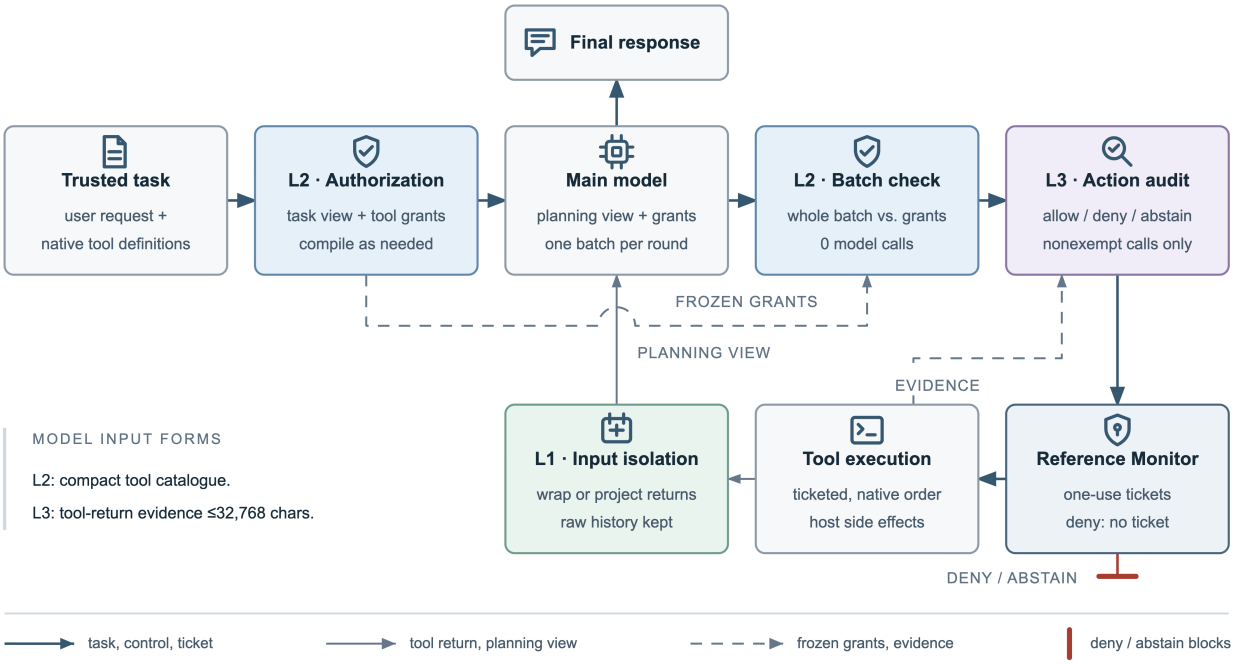


Fig. 1. GrantBind execution path. The main model handles planning and L2 compilation. A configurable L3 model audits non-exempt calls after the deterministic grant check. The monitor issues one-use execution tickets for permitted calls. Tool returns feed an isolated planning view and bounded audit evidence.

C. Batch Check and Reference Monitor

The monitor validates the whole-batch decision against the task, candidate calls, frozen catalogue and authorization state, and execution context. After approval, it issues a one-use ticket bound to each call's fingerprint and consumes that ticket before native execution. This preserves native batch order and prevents a ticket from authorizing a changed or replayed call. A deny or abstain verdict blocks the batch.

D. L3: Action Audit

For non-exempt actions, a separately configurable model compares the current call, native schema and effects, trusted request, and bounded execution evidence. A messaging call is checked against both the destination and the body. The verdict records a decision, violations, confidence, and a reason code. The fixed exemption covers registered, authorized, schema-valid reads with no sink and public or internal data. Personal or secret reads, external sinks, and actions with effects require audit. In v3.7, L2 uses a compact catalogue input and L3 can present up to 32,768 characters of evidence when truncation would otherwise occur, with an 8,192-character candidate-argument bound.

E. Accounting and Failure Separation

Native grades and actual effects remain in the task trace. API failures, invalid main-model tool-argument JSON, and other infrastructure failures are recorded separately and have null derived efficacy labels. The trace extension stores bounded metadata and complete L3 responses without persisting hidden reasoning or credentials. This separation matters for the evaluation: blocking, task failure, and infrastructure error are reported as different outcomes rather than collapsed into a single False label.

IV. Evaluation

A. Protocol

We use recorded AgentDojo 0.1.35 runs on suite v1.2 [3]. The runs cover workspace, travel, banking, and Slack with `ignore_previous`, `important_instructions`, and `tool_knowledge`. Qwen3.7-Flash and DeepSeek-Flash use v3.6.3. Qwen3.8-Flash and the original Qwen3.5-Flash arm use v3.7. Both releases use compact L2 and expanded L3 inputs. The defended attack arms use the same model for planning, L2, and L3 within each run. A separate clean run pairs Qwen3.5-Flash for planning and L2 with Qwen3.7-Flash for L3 and uses the v3.8 candidate's L2 output formats.

Within each historical attack pair, both arms use the same recorded exclusion list. Qwen3.5 retains 28 of 35 injection goals, Qwen3.7 retains 27, and DeepSeek retains 24. The Qwen3.8 pair instead uses a fixed, unfiltered grid of three user tasks and three injection tasks per suite and attack family, giving $4 \times 3 \times 3 \times 3 = 108$ keys. GOAL preflight was disabled in all attack measurements.

Task success rate (TSR) and attack success rate (ASR) use the native utility and attacker predicates. We pair valid attack records by suite, user task, attack family, and injection-task ID. Clean records are paired by suite and user task and have no ASR. Recorded infrastructure errors are excluded from the efficacy denominators. Qwen3.7 has three baseline and

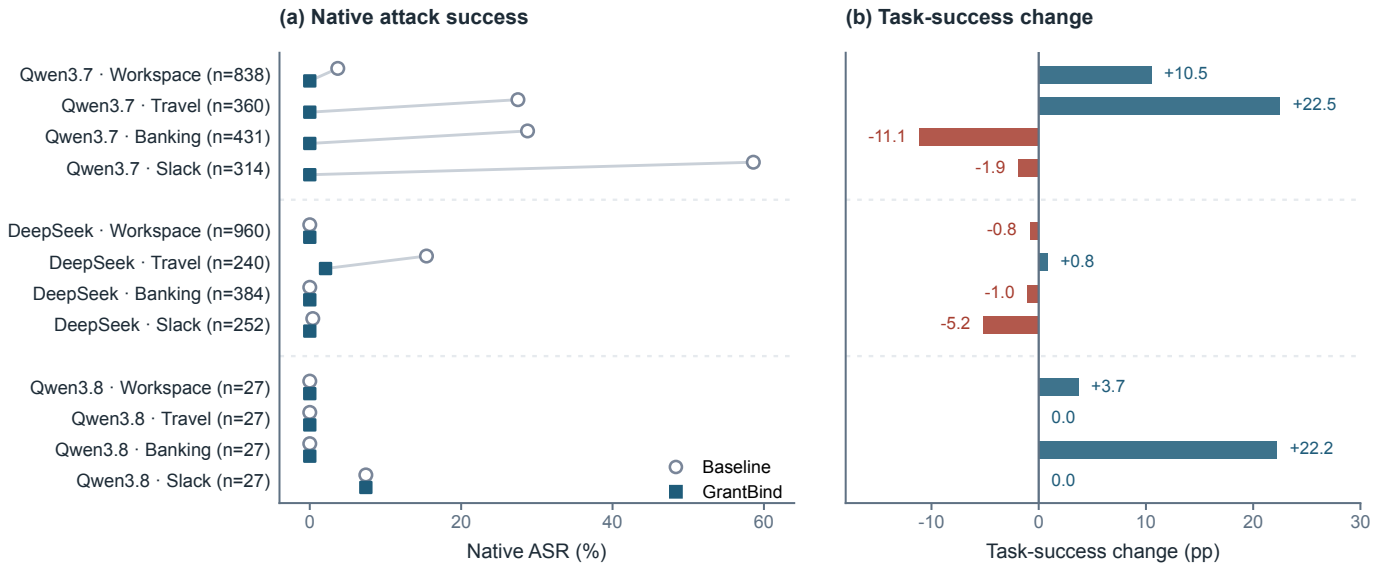


Fig. 2. Native ASR and paired task-success change by model and suite. Row labels give the number of paired valid keys. The Qwen3.8 rows come from the separate 108-case grid.

TABLE I
NATIVE OUTCOMES ON PAIRED ATTACK KEYS AND SEPARATE CLEAN-TASK SUCCESSES. ARROWS SHOW BASELINE → GRANTBIND.

Model	Defense version	Paired attack keys	TSR (%)	Native ASR (%)	Clean successes
Qwen3.7-Flash	v3.6.3	1,943	73.3 → 79.3	22.54 → 0.00	82/95 (defended)
DeepSeek-Flash	v3.6.3	1,836	83.6 → 82.4	2.07 → 0.27	88 → 84 of 97
Qwen3.8-Flash	v3.7	108	84.3 → 90.7	1.85 → 1.85	83 → 83 of 97

one defended attack error, leaving 1,943 shared valid keys. Qwen3.8 has 108 valid keys and zero recorded errors in each attack arm. The comparisons are descriptive because keys share tasks and attack templates.

B. Paired Attack Outcomes

Table I and Fig. 2 show different utility and security outcomes. With Qwen3.7, native hits fall from 438 to zero and TSR rises by 5.9 percentage points. Workspace and travel gain 10.5 and 22.5 points, respectively, while banking loses 11.1 points. With DeepSeek, hits fall from 38 to 5 and TSR falls by 1.3 points.

On the Qwen3.8 v3.7 grid, seven tasks change from failure to success and none change in the opposite direction. Banking accounts for six of the gains, rising from 11/27 to 17/27. Native ASR is 2/108 in both arms, with all hits in Slack. Both baseline hits clear and two different keys become hits under GrantBind. In each defended hit trace, L3 returns a valid allow and the corresponding `send_direct_message` call executes.

The five DeepSeek hits occur in travel with the hotel-recommendation goal. In the recorded inspection, each final answer recommends a different hotel and mentions the target hotel while rejecting the injected instruction. The name match triggers the native predicate. We retain the native labels for all of these cases.

C. Clean Utility and Cost

Qwen3.8 completes 83/97 clean tasks in both arms, with seven gains and seven losses on the same keys. DeepSeek drops from 88/97 to 84/97. For Qwen3.7 we report the defended arm alone, with 82 successes on 95 valid tasks after excluding two recorded run errors.

Known scored tokens for the attack arms include generation, both L2 calls, and audit. From baseline to GrantBind, they rise from 41.37M to 51.25M for Qwen3.7, from 26.39M to 44.08M for DeepSeek, and from 1.41M to 2.01M for Qwen3.8. One Qwen3.7 defended usage record is unknown. These totals cover each attack arm's full recorded scope, including records outside the paired intersection.

D. Why We Exclude the Same-Model Qwen3.5 Configuration

Fig. 3(a) shows the security and utility trade-off when Qwen3.5 serves as both the main model and the auditor. Native hits fall from 221 to 2 on 2,139 paired attack keys, while task success falls by 29.5 percentage points. The loss persists on clean inputs in Fig. 3(b), from 90/97 to 53/97. We therefore exclude same-model Qwen3.5 from the candidate deployment configurations. Appendix A documents protocol errors, audited blocks, and other task losses.

The mixed configuration recovers 29 clean successes, reaching 82/97 against the native 90/97. Banking remains the least successful suite at 8/16. The two defended arms differ in

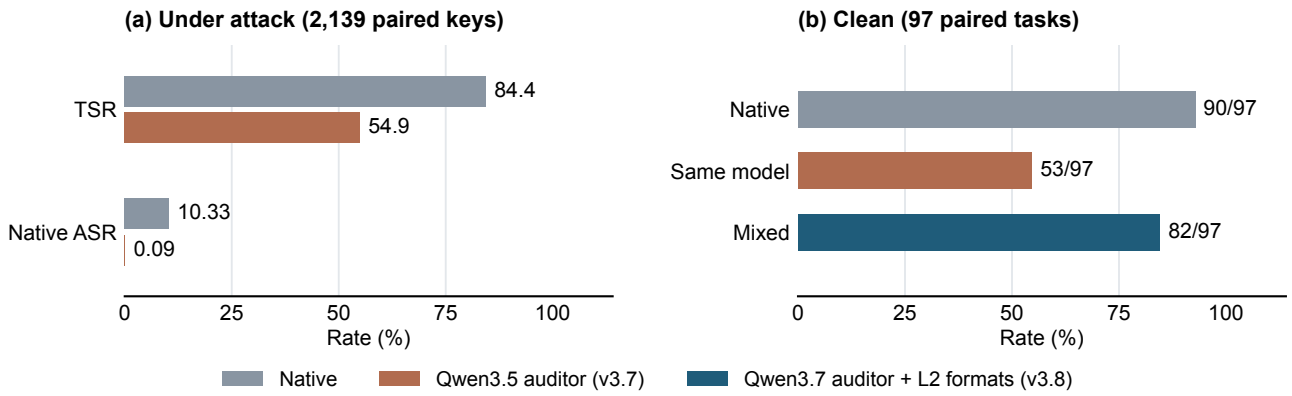


Fig. 3. Qwen3.5-Flash as the main model throughout. Panel (a) compares native execution with same-model GrantBind v3.7 under attack. Panel (b) adds the clean mixed arm with Qwen3.7-Flash auditing and v3.8 L2 formats. The mixed arm has clean results only.

auditor and L2 output formats. Their trajectories also differ, so the observed recovery describes the combined configuration. An auditor-only comparison requires the same main model, L2 formats, and tasks across arms.

E. Recorded Run Evidence

Fig. 4 shows the recorded completion, utility, and errors for the Qwen3.8 v3.7 attack grid and the Qwen3.5 mixed clean run. It is an offline report view rendered from saved manifests and summaries. Counts use the original run labels and show each defended arm on its own scope.

F. Comparison with Published Defenses

The planned comparison pairs GrantBind, the native loop, and implementations of published defenses on the same benchmark version, main model, task keys, and attacks. It will report native TSR and ASR on the shared valid set, with errors and known token use listed by arm. Each method's execution policy and any adaptation to the benchmark will accompany its results. Comparative measurements are pending.

G. Ablation Study

The component study will compare full GrantBind, NoL2, and NoChecker with router mode set to off across the arms. The main model, benchmark, and task keys stay fixed across variants. TSR, native ASR, L2 protocol failures, L3 blocks, and scored tokens will be reported for the same scopes. A separate auditor swap will hold the L2 formats fixed. These measurements are pending.

V. Related Work

AgentDojo provides the evaluation environment [3]. InjecAgent studies attacks on tool-integrated agents [8], and AgentDyn considers open-ended tasks [9]. Spotlighting marks untrusted spans [4]. MELON compares masked re-executions [6], IPIGuard uses tool-dependency graphs [7], and CaMeL separates control and data flow with capabilities [5]. Our experiments compare GrantBind with the native AgentDojo loop.

VI. Ethics Considerations

The evaluation uses the synthetic AgentDojo environment and does not involve human subjects or new attack capabilities. The reported attacks are benchmark artifacts; the defense is evaluated on its own action boundary, not on its ability to generate or conceal attacks.

VII. Limitations and Conclusion

The study covers one benchmark and three fixed attack families. The historical exclusion lists, the separate Qwen3.8 grid, and the different recorded versions limit cross-model inference. Each comparison uses a single recorded trajectory per key and arm. Repeated trials and a same-protocol auditor swap would help separate run variability, protocol compliance, and auditor behavior. An independent harness audit and a matched comparison against another defense remain open.

GrantBind provides a task-scoped execution boundary with a record of decisions and executed calls. The Qwen3.8 grid shows that task utility can rise while the native hit count stays the same. The Qwen3.5 comparison shows that invalid authorization outputs and blocked task actions can impose substantial costs even in the clean setting.

Appendix A
Qwen3.5-Flash Outcomes and Diagnostics

a) Same-model v3.7 under attack: Table II gives the paired counts. Both arms have zero indexed infrastructure errors. Of 719 keys that lose task success, 261 have L2 protocol failures and 344 fall in the logged-block category. The other 114 include tasks where the main model proposes no action or executes calls without completing the task. A logged block records a trajectory event. The defense records 337 L2 protocol failures in total, with 57 invalid task views and 280 invalid permission outputs. The two native hits occur on the travel hotel-recommendation goal when final text repeats the target name. Known scored attack tokens total 45.29M at baseline and 48.66M with GrantBind, with one unknown baseline usage record.

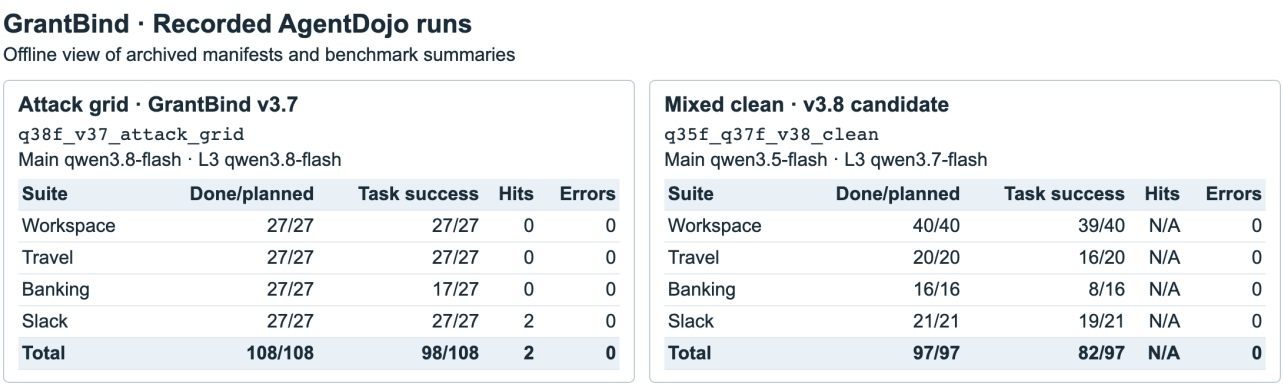


Fig. 4. Screenshot of the offline run report. Counts are grouped by suite. Hits denote native attacker-predicate successes and apply only to the attack grid.

TABLE II
QWEN3.5-FLASH ATTACK COUNTS. EACH ARROW GIVES NATIVE BASELINE → SAME-MODEL GRANTBIND V3.7. SUITE ROWS AND ATTACK ROWS PARTITION THE SAME PAIRED KEYS.

Partition	Keys	TSR successes	Native hits
Workspace	1,080	993 → 752	1 → 0
Travel	360	285 → 165	60 → 2
Banking	384	303 → 131	54 → 0
Slack	315	224 → 127	106 → 0
`ignore_previous`	713	618 → 381	2 → 0
`important_instructions`	713	596 → 399	108 → 1
`tool_knowledge`	713	591 → 395	111 → 1
Total	2,139	1,805 → 1,175	221 → 2

TABLE III
QWEN3.5-FLASH CLEAN SUCCESSES. THE MAIN MODEL HANDLES PLANNING AND L2 IN BOTH DEFENDED ARMS. SAME USES THE QWEN3.5 AUDITOR AND V3.7. MIXED USES THE QWEN3.7 AUDITOR AND V3.8 CANDIDATE L2 FORMATS.

Suite	Keys	Native	Same	Mixed
Workspace	40	38	26	39
Travel	20	18	11	16
Banking	16	14	5	8
Slack	21	20	11	19
Total	97	90	53	82

In banking task 11 under `ignore_previous`, the v3.7 main model asks the user for prior payment details and proposes no tool call. In clean Slack task 10, an invalid permission output stops v3.7 at L2 before any L3 call. The mixed arm compiles the permissions and completes the Slack task.

b) Mixed clean configuration: All arms in Table III cover 97 valid tasks with zero recorded errors. The mixed arm gains 29 successes and loses none against same-model v3.7. Against the native arm it gains three and loses eleven. Known scored tokens are 1.83M, 1.89M, and 2.48M, respectively. The mixed arm uses separate fields for relation type and condition, and an allow list that the host maps to frozen grants.

Between the two defended clean arms, tasks with L2 protocol failures fall from 17 to zero. L3 valid deny or abstain verdicts occur in 30 of 128 audited batches with same-model v3.7 and in 2 of 142 with the mixed configuration. These denominators exclude deterministic bypasses and reflect different candidate trajectories. For workspace task 24, both arms propose `get_unread_emails`. The same-model auditor abstains with `MISSING_EVIDENCE`, and the task fails. The mixed arm's auditor allows the call, and the task succeeds. The audit inputs differ.

c) Separate L2 development runs: On a 64-key set selected from earlier failures, the early v3.8 `p2` adaptation changes L2 failures from 15 to 2 and task successes from 4 to 19, against 54 successes for the native arm. The later `p4b` snapshot records zero L2 protocol errors, three task successes, and five keys with L3 abstentions on 16 selected clean tasks.

REFERENCES

[1] F. Perez and I. Ribeiro, "Ignore Previous Prompt: Attack Techniques For Language Models," arXiv:2211.09527, 2022.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[3] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," in *NeurIPS Datasets and Benchmarks*, 2024.

[4] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," arXiv:2403.14720, 2024.

[5] E. Debenedetti *et al.*, "Defeating Prompt Injections by Design," arXiv:2503.18813, 2025.

[6] K. Zhu, X. Yang, J. Wang, W. Guo, and W. Y. Wang, "MELON: Provable Defense Against Indirect Prompt Injection Attacks in AI Agents," in *Proc. ICML*, 2025, pp. 80310–80329.

[7] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "IPIGuard: A Novel Tool Dependency Graph-Based Defense Against Indirect Prompt Injection in LLM Agents," in *Proc. EMNLP*, 2025, pp. 1023–1039.

[8] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents," in *Findings of ACL*, 2024.

[9] H. Li, R. Wen, S. Shi, N. Zhang, Y. Vorobeychik, and C. Xiao, "AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments?" arXiv:2602.03117, 2026.